

Problem Statement

Business context

Employee Promotion means the ascension of an employee to higher ranks, this aspect of the job is what drives employees the most. The ultimate reward for dedication and loyalty towards an organization and the HR team plays an important role in handling all these promotion tasks based on ratings and other attributes available.

The HR team in JMD company stored data on the promotion cycle last year, which consists of details of all the employees in the company working last year and also if they got promoted or not, but every time this process gets delayed due to so many details available for each employee - it gets difficult to compare and decide.

Objective

For the upcoming appraisal cycle, the HR team wants to utilize the stored data and leverage machine learning to make a model that will predict if a person is eligible for promotion or not. You, as a data scientist at JMD company, need to come up with the best possible model that will help the HR team to predict if a person is eligible for promotion or not.

Data Description

- employee_id: Unique ID for the employee
- department: Department of employee
- region: Region of employment (unordered)
- education: Education Level
- gender: Gender of Employee
- recruitment_channel: Channel of recruitment for employee
- no_of_trainings: no of other training completed in the previous year on soft skills, technical skills, etc.
- age: Age of Employee
- previous_year_rating: Employee Rating for the previous year
- length_of_service: Length of service in years
- awards_won: if awards won during the previous year then 1 else 0
- avg_training_score: Average score in current training evaluations
- is_promoted: (Target) Recommended for promotion

Please read the instructions carefully before starting the project.

This is a commented Jupyter IPython Notebook file in which all the instructions and tasks to be performed are mentioned.

- Blanks '_____' are provided in the notebook that needs to be filled with an appropriate code to get the correct result. With every '_____' blank, there is a comment that briefly describes what needs to be filled in the blank space.
- Identify the task to be performed correctly, and only then proceed to write the required code.
- Fill the code wherever asked by the commented lines like "# write your code here" or "# complete the code". Running incomplete code may throw error.
- Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors.
- Add the results/observations (wherever mentioned) derived from the analysis in the presentation and submit the same.

Importing necessary libraries

```
# Installing the Libraries with the specified version.
# uncomment and run the following line if Google Colab is being used
# !pip install scikit-learn==1.2.2 seaborn==0.13.1 matplotlib==3.7.1
# numpy==1.25.2 pandas==1.5.3 imbalanced-learn==0.10.1 xgboost==2.0.3
# -q --user

# Installing the Libraries with the specified version.
# uncomment and run the following lines if Jupyter Notebook is being used
# !pip install scikit-learn==1.2.2 seaborn==0.13.1 matplotlib==3.7.1
# numpy==1.25.2 pandas==1.5.3 imbalanced-learn==0.10.1 xgboost==2.0.3
# -q --user
# !pip install --upgrade -q threadpoolctl
```

```

# Libraries to help with reading and manipulating data
import pandas as pd
import numpy as np

# Libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# To tune model, get different metric scores, and split data
from sklearn.metrics import (
    f1_score,
    accuracy_score,
    recall_score,
    precision_score,
    confusion_matrix,
    roc_auc_score,
    ConfusionMatrixDisplay,
)
from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_val_score

# To be used for data scaling and one hot encoding
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder

# To oversample and undersample data
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

# To do hyperparameter tuning
from sklearn.model_selection import RandomizedSearchCV

# To impute missing values
from sklearn.impute import SimpleImputer

# To define maximum number of columns to be displayed in a dataframe
pd.set_option("display.max_columns", None)

# To suppress scientific notations for a dataframe
pd.set_option("display.float_format", lambda x: "%.3f" % x)

# To help with model building
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
    BaggingClassifier,
)
from xgboost import XGBClassifier

# To suppress scientific notations
pd.set_option("display.float_format", lambda x: "%.3f" % x)

# To suppress warnings
import warnings

warnings.filterwarnings("ignore")

```

Loading the dataset

```

from google.colab import drive
drive.mount('/content/drive')

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```

promotion = pd.read_csv("employee_promotion.csv")

```

Data Overview

The initial steps to get an overview of any dataset is to:

- observe the first few rows of the dataset, to check whether the dataset has been loaded properly or not
- get information about the number of rows and columns in the dataset

- find out the data types of the columns to ensure that data is stored in the preferred format and the value of each property is as expected.
- check the statistical summary of the dataset to get an overview of the numerical columns of the data

Checking the shape of the dataset

```
# Checking the number of rows and columns in the training data
promotion.shape ## Complete the code to view dimensions of the train data
```

```
(54808, 13)
```

```
# Let's create a copy of the data
data = promotion.copy()
```

Displaying the first few rows of the dataset

```
# Let's view the first 5 rows of the data
data.head() ## Complete the code to view top 5 rows of the data
```

	employee_id	department	region	education	gender	recruitment_channel	no_of_trainings	age	previous
0	65438	Sales & Marketing	region_7	Master's & above	f	sourcing	1	35	5.000
1	65141	Operations	region_22	Bachelor's	m	other	1	30	5.000
2	7513	Sales & Marketing	region_19	Bachelor's	m	sourcing	1	34	3.000
3	2542	Sales & Marketing	region_23	Bachelor's	m	other	2	39	1.000
4	48945	Technology	region_26	Bachelor's	m	other	1	45	3.000

```
# Let's view the last 5 rows of the data
data.tail() ## Complete the code to view last 5 rows of the data
```

	employee_id	department	region	education	gender	recruitment_channel	no_of_trainings	age	pre
54803	3030	Technology	region_14	Bachelor's	m	sourcing	1	48	3.00
54804	74592	Operations	region_27	Master's & above	f	other	1	37	2.00
54805	13918	Analytics	region_1	Bachelor's	m	other	1	27	5.00
54806	13614	Sales & Marketing	region_9	NaN	m	sourcing	1	29	1.00
54807	51526	HR	region_22	Bachelor's	m	other	1	27	1.00

Checking the data types of the columns for the dataset

```
# Let's check the data types of the columns in the dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54808 entries, 0 to 54807
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   employee_id            54808 non-null  int64
1   department             54808 non-null  object
2   region                 54808 non-null  object
3   education              52399 non-null  object
4   gender                 54808 non-null  object
5   recruitment_channel     54808 non-null  object
6   no_of_trainings        54808 non-null  int64
7   age                    54808 non-null  int64
8   previous_year_rating   50684 non-null  float64
9   length_of_service      54808 non-null  int64
10  awards_won             54808 non-null  int64
11  avg_training_score     52248 non-null  float64
12  is_promoted            54808 non-null  int64
dtypes: float64(2), int64(6), object(5)
memory usage: 5.4+ MB
```

Checking for duplicate values

```
# Let's check for duplicate values in the data
data.duplicated().sum() ## Complete the code to check duplicate entries in
the data

np.int64(0)
```

Checking for missing values

```
# Let's check for missing values in the data
data.isnull().sum() ## Complete the code to check missing entries in the
train data
```

	0	
employee_id	0	
department	0	
region	0	
education	2409	
gender	0	
recruitment_channel	0	
no_of_trainings	0	
age	0	
previous_year_rating	4124	
length_of_service	0	
awards_won	0	
avg_training_score	2560	
is_promoted	0	

dtype: int64

Statistical summary of the dataset

```
# Let's view the statistical summary of the numerical columns in the data
data.describe() ## Complete the code to print the statistical summary of the
train data
```

	employee_id	no_of_trainings	age	previous_year_rating	length_of_service	awards_won	avg_train
count	54808.000	54808.000	54808.000	50684.000	54808.000	54808.000	52248.000
mean	39195.831	1.253	34.804	3.329	5.866	0.023	63.712
std	22586.581	0.609	7.660	1.260	4.265	0.150	13.522
min	1.000	1.000	20.000	1.000	1.000	0.000	39.000
25%	19669.750	1.000	29.000	3.000	3.000	0.000	51.000
50%	39225.500	1.000	33.000	3.000	5.000	0.000	60.000
75%	58730.500	1.000	39.000	4.000	7.000	0.000	77.000
max	78298.000	10.000	60.000	5.000	37.000	1.000	99.000

```
# Let's view the statistical summary of the numerical columns in the data
data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
employee_id	54808.000	39195.831	22586.581	1.000	19669.750	39225.500	58730.500	78298.000
no_of_trainings	54808.000	1.253	0.609	1.000	1.000	1.000	1.000	10.000
age	54808.000	34.804	7.660	20.000	29.000	33.000	39.000	60.000
previous_year_rating	50684.000	3.329	1.260	1.000	3.000	3.000	4.000	5.000
length_of_service	54808.000	5.866	4.265	1.000	3.000	5.000	7.000	37.000
awards_won	54808.000	0.023	0.150	0.000	0.000	0.000	0.000	1.000
avg_training_score	52248.000	63.712	13.522	39.000	51.000	60.000	77.000	99.000
is_promoted	54808.000	0.085	0.279	0.000	0.000	0.000	0.000	1.000

Let's check the number of unique values in each column

```
data.nunique()
```

	0
employee_id	54808
department	9
region	34
education	3
gender	2
recruitment_channel	3
no_of_trainings	10
age	41
previous_year_rating	5
length_of_service	35
awards_won	2
avg_training_score	59
is_promoted	2

dtype: int64

```
for i in data.describe(include=["object"]).columns:
    print("Unique values in", i, "are :")
    print(data[i].value_counts())
    print("*" * 50)
```

Unique values in department are :

```
department
Sales & Marketing    16840
Operations            11348
Technology            7138
Procurement          7138
Analytics             5352
Finance               2536
HR                   2418
Legal                1039
R&D                   999
```

Name: count, dtype: int64

Unique values in region are :

```
region
region_2    12343
region_22   6428
region_7    4843
region_15   2808
region_13   2648
region_26   2260
region_31   1935
region_4    1703
region_27   1659
region_16   1465
region_28   1318
region_11   1315
region_23   1175
region_29    994
region_32    945
region_19    874
region_20    850
region_14    827
region_25    819
region_17    796
region_5     766
region_6     690
region_30    657
region_8     655
region_10    648
region_1     610
region_24    508
region_12    500
region_9     420
region_21    411
region_3     346
region_34    292
```

```

region_33      269
region_18      31
Name: count, dtype: int64
*****

Unique values in education are :
education
Bachelor's      36669
Master's & above  14925
Below Secondary   805
Name: count, dtype: int64
*****

Unique values in gender are :
gender
m      38496
f      16312
Name: count, dtype: int64
*****

Unique values in recruitment_channel are :
recruitment_channel
other      30446
sourcing   23220
referred    1142
Name: count, dtype: int64
*****

```

```

# ID column consists of uniques ID for clients and hence will not add value
  to the modeling
data.drop(columns="employee_id", inplace=True)

```

```
data["is_promoted"].value_counts(1)
```

	proportion
is_promoted	
0	0.915
1	0.085

```
dtype: float64
```

Exploratory Data Analysis

The below functions need to be defined to carry out the Exploratory Data Analysis.

```
# function to plot a boxplot and a histogram along the same scale.
```

```

def histogram_boxplot(data, feature, figsize=(12, 7), kde=False, bins=None):
    """
    Boxplot and histogram combined

    data: dataframe
    feature: dataframe column
    figsize: size of figure (default (12,7))
    kde: whether to the show density curve (default False)
    bins: number of bins for histogram (default None)
    """
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2, # Number of rows of the subplot grid= 2
        sharex=True, # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    ) # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
    ) # boxplot will be created and a triangle will indicate the mean value
      of the column
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins,
        palette="winter"
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    ) # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    ) # Add mean to the histogram
    ax_hist2.axvline(

```

```

        data[feature].median(), color="black", linestyle="-"
    ) # Add median to the histogram

# function to create labeled barplots

def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all
        levels)
    """

    total = len(data[feature]) # Length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 1, 5))
    else:
        plt.figure(figsize=(n + 1, 5))

    plt.xticks(rotation=90, fontsize=15)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n].sort_values(),
    )

    for p in ax.patches:
        if perc == True:
            label = "{:.1f}%".format(
                100 * p.get_height() / total
            ) # percentage of each class of the category
        else:
            label = p.get_height() # count of each level of the category

        x = p.get_x() + p.get_width() / 2 # width of the plot
        y = p.get_height() # height of the plot

        ax.annotate(
            label,
            (x, y),
            ha="center",
            va="center",
            size=12,
            xytext=(0, 5),
            textcoords="offset points",
        ) # annotate the percentage

    plt.show() # show the plot

# function to plot stacked bar chart

def stacked_barplot(data, predictor, target):
    """
    Print the category counts and plot a stacked bar chart

    data: dataframe
    predictor: independent variable
    target: target variable
    """

    count = data[predictor].nunique()
    sorter = data[target].value_counts().index[-1]
    tab1 = pd.crosstab(data[predictor], data[target],
        margins=True).sort_values(
            by=sorter, ascending=False
        )
    print(tab1)
    print("-" * 120)
    tab = pd.crosstab(data[predictor], data[target],
        normalize="index").sort_values(
            by=sorter, ascending=False
        )
    tab.plot(kind="bar", stacked=True, figsize=(count + 1, 5))
    plt.legend(

```

```

        loc="lower left", frameon=False,
    )
    plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
    plt.show()

### Function to plot distributions

def distribution_plot_wrt_target(data, predictor, target):

    fig, axs = plt.subplots(2, 2, figsize=(12, 10))

    target_uniq = data[target].unique()

    axs[0, 0].set_title("Distribution of target for target=" +
                        str(target_uniq[0]))
    sns.histplot(
        data=data[data[target] == target_uniq[0]],
        x=predictor,
        kde=True,
        ax=axs[0, 0],
        color="teal",
    )

    axs[0, 1].set_title("Distribution of target for target=" +
                        str(target_uniq[1]))
    sns.histplot(
        data=data[data[target] == target_uniq[1]],
        x=predictor,
        kde=True,
        ax=axs[0, 1],
        color="orange",
    )

    axs[1, 0].set_title("Boxplot w.r.t target")
    sns.boxplot(data=data, x=target, y=predictor, ax=axs[1, 0],
                palette="gist_rainbow")

    axs[1, 1].set_title("Boxplot (without outliers) w.r.t target")
    sns.boxplot(
        data=data,
        x=target,
        y=predictor,
        ax=axs[1, 1],
        showfliers=False,
        palette="gist_rainbow",
    )

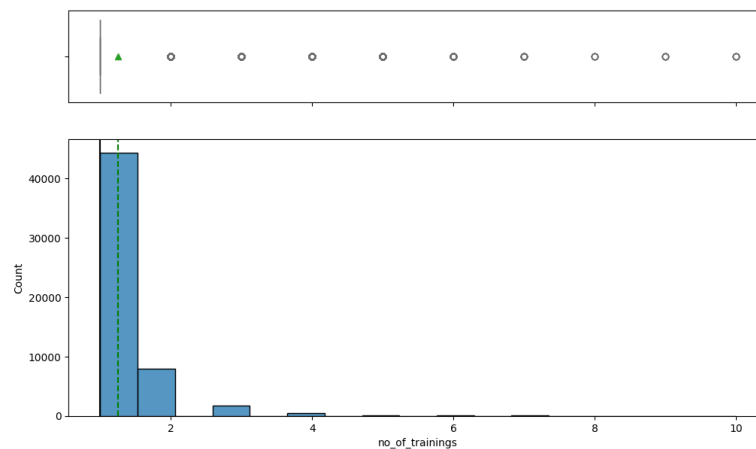
    plt.tight_layout()
    plt.show()

```

Univariate analysis

Observations on No. of Trainings

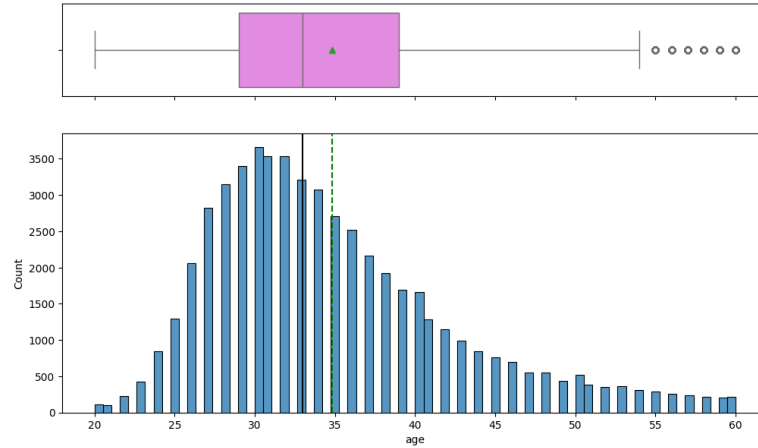
```
histogram_boxplot(data, "no_of_trainings")
```



Let's see the distribution of age of employee

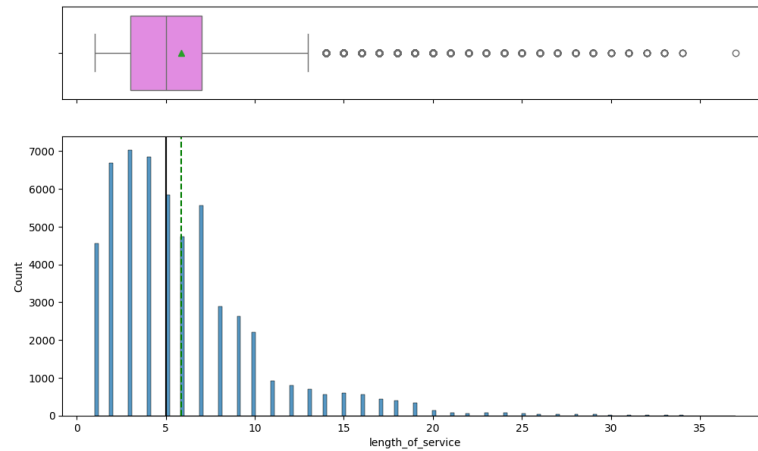
Observations on Age

```
histogram_boxplot(data, "age") ## Complete the code to create  
    histogram_boxplot for 'age'
```



Observations on Length of Service

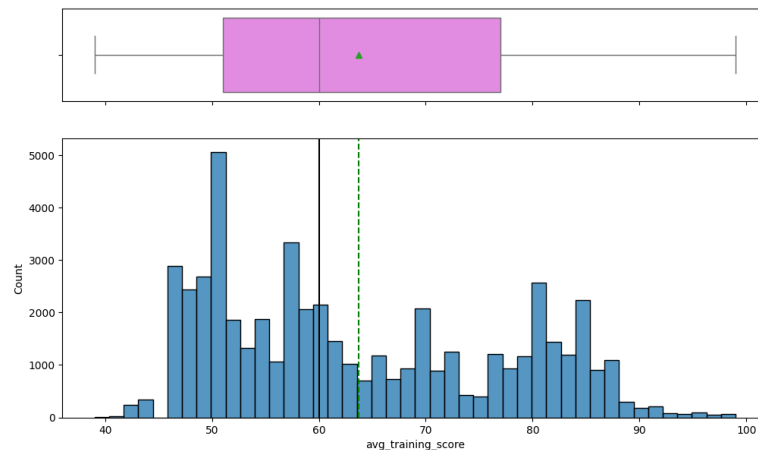
```
histogram_boxplot(data, "length_of_service") ## Complete the code to create  
    histogram_boxplot for 'length_of_service'
```



Let's see the distribution of average training score of employee

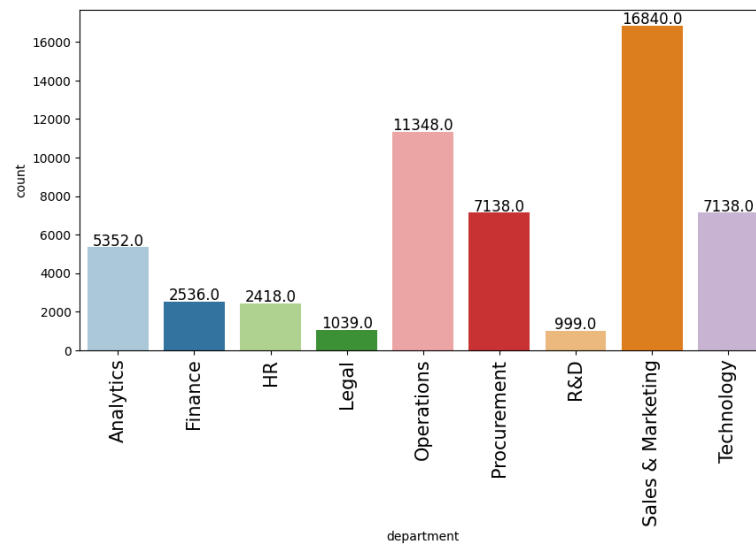
Observations on Average Training Score

```
histogram_boxplot(data, "avg_training_score") ## Complete the code to create  
    histogram_boxplot for 'avg_training_score'
```



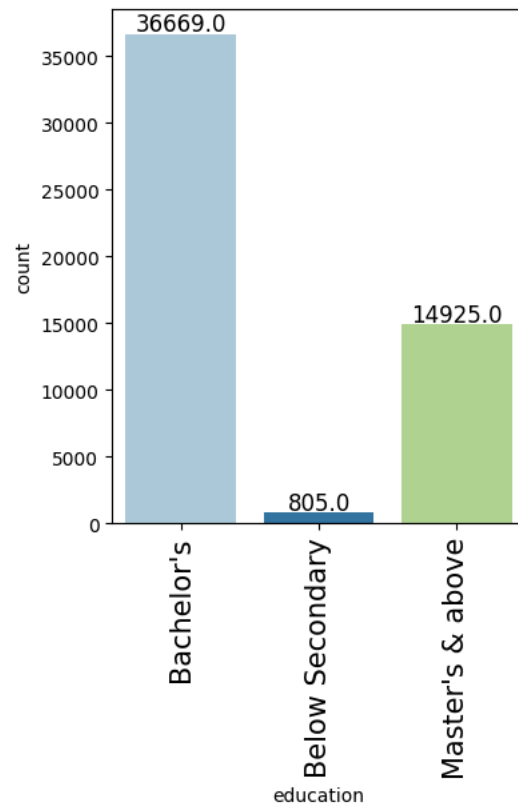
Observations on Department

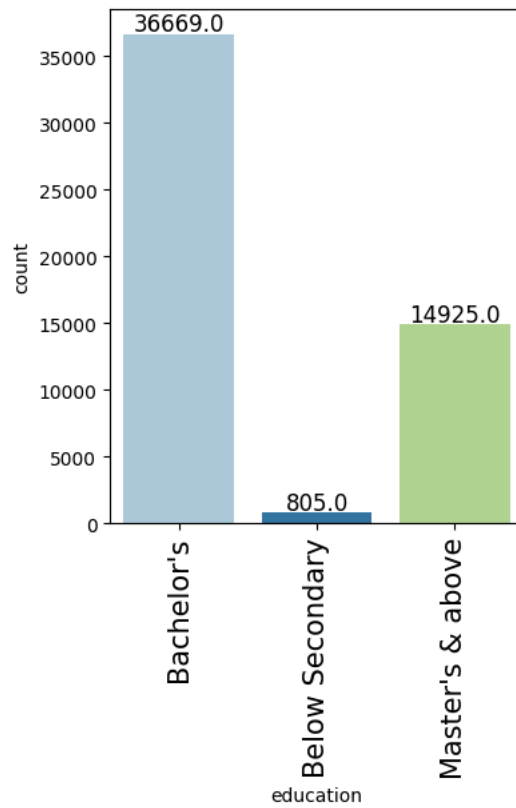
```
labeled_barplot(data, "department")
```



Observations on Education

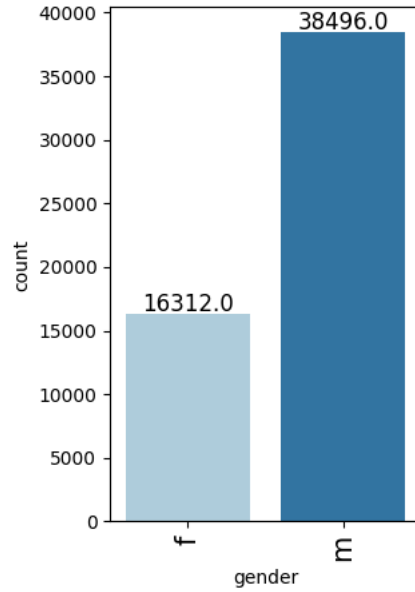
```
labeled_barplot(data, "education") ## Complete the code to create  
labeled_barplot for 'education'
```





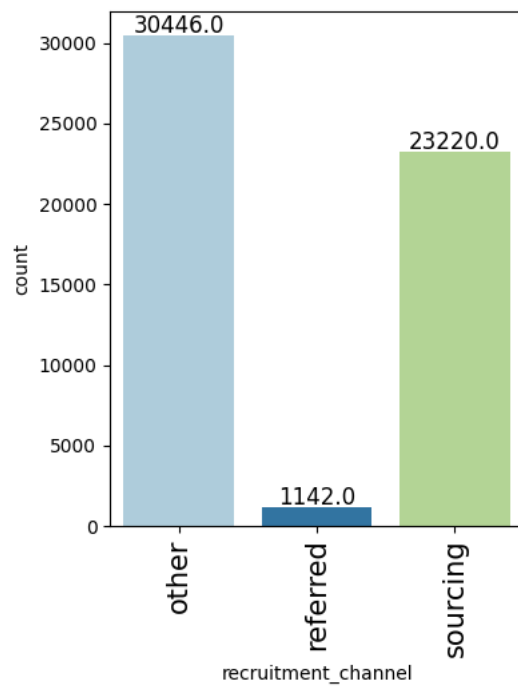
Observations on Gender

```
labeled_barplot(data, "gender") ## Complete the code to create  
labeled_barplot for 'gender'
```



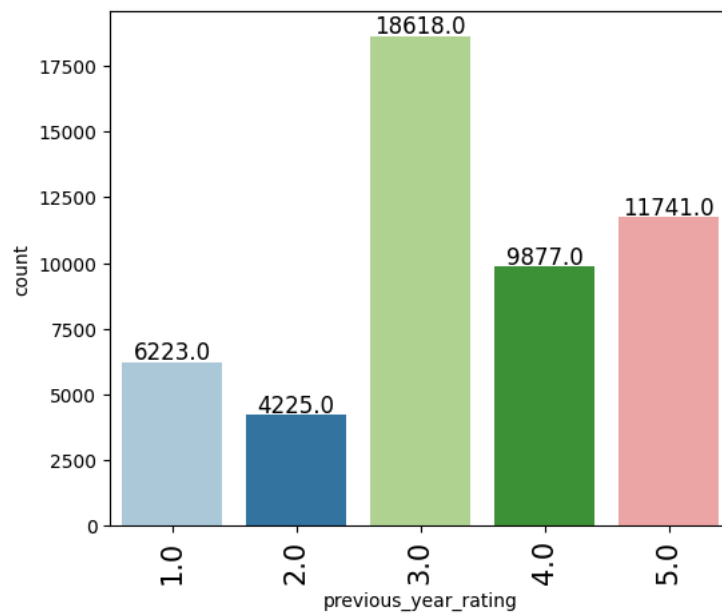
Observations on Recruitment Channel

```
labeled_barplot(data, "recruitment_channel") ## Complete the code to create  
labeled_barplot for 'recruitment_channel'
```



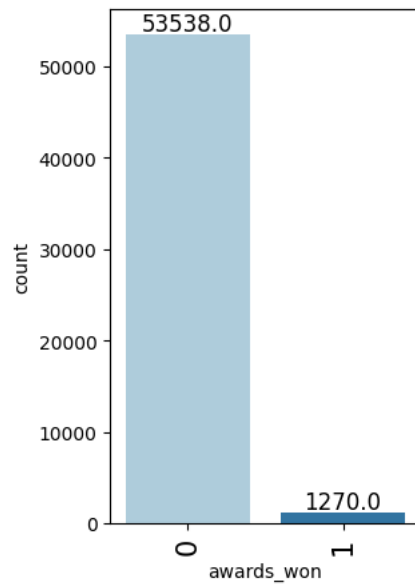
Observations on Previous Year Rating

```
labeled_barplot(data, "previous_year_rating") ## Complete the code to create  
labeled_barplot for 'previous_year_rating'
```



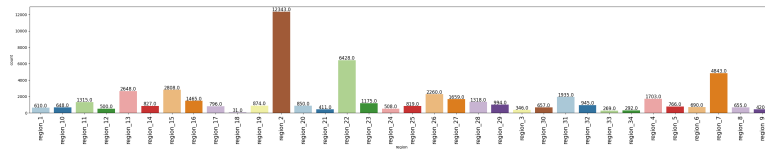
Observations on Awards Won

```
labeled_barplot(data, "awards_won") ## Complete the code to create  
labeled_barplot for 'awards_won'
```



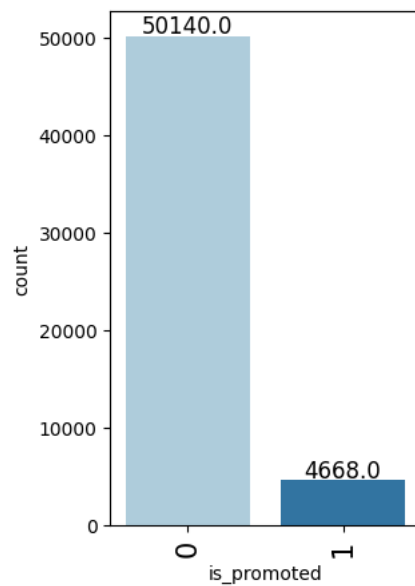
Observations on Region

`labeled_barplot(data, "region")` *## Complete the code to create labeled_barplot for 'region'*



Observations on target variable

`labeled_barplot(data, "is_promoted")` *## Complete the code to create labeled_barplot for 'is_promoted'*



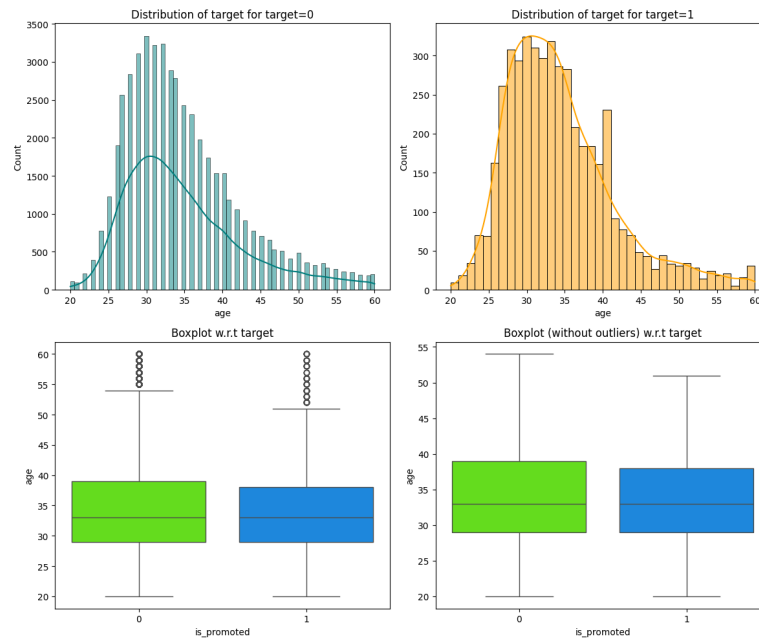
Bivariate Analysis

`sns.pairplot(data, hue="is_promoted")`
 <seaborn.axisgrid.PairGrid at 0x78bb8149e210>



Target variable vs Age

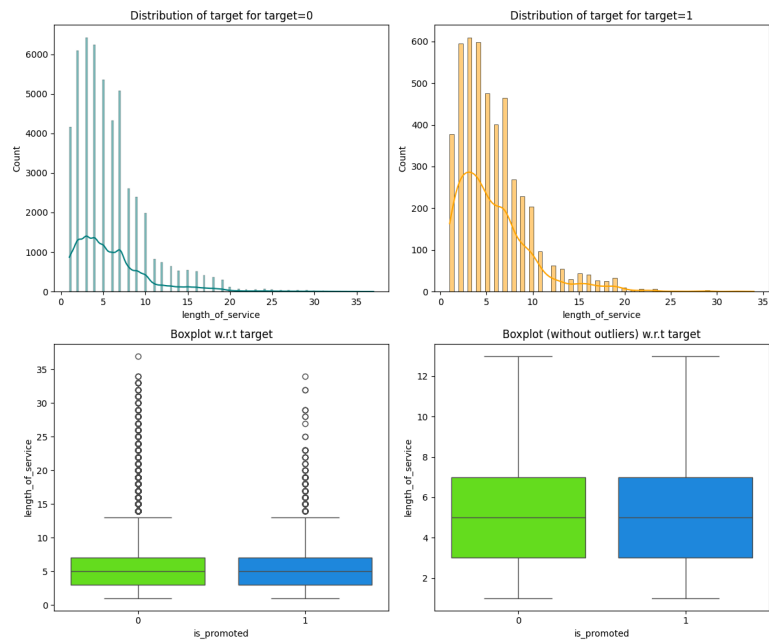
`distribution_plot_wrt_target(data, "age", "is_promoted")`



Let's see the change in length of service (length_of_service) vary by the employee's promotion status (is_promoted)?

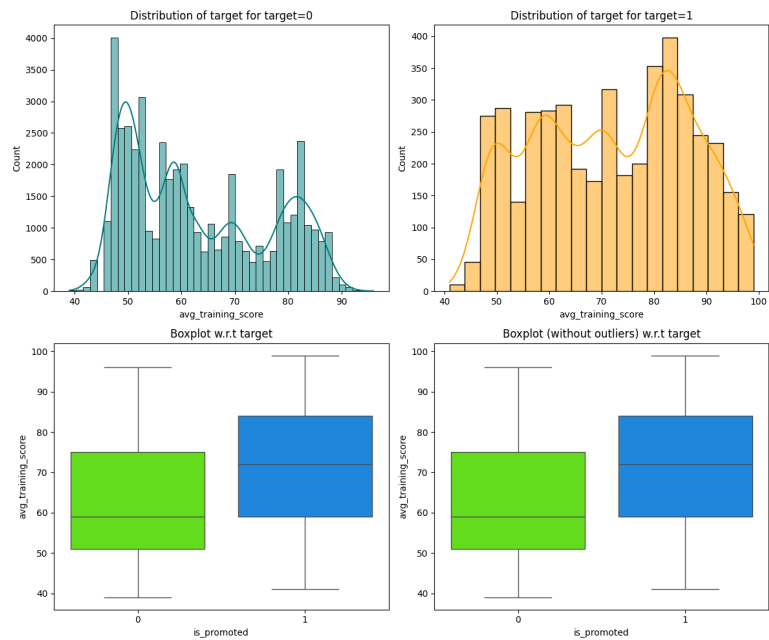
Target variable vs Length of Service

`distribution_plot_wrt_target(data, "length_of_service", "is_promoted") ##`
Complete the code to create distribution_plot for length_of_service vs is_promoted



Target variable vs Average Training Score

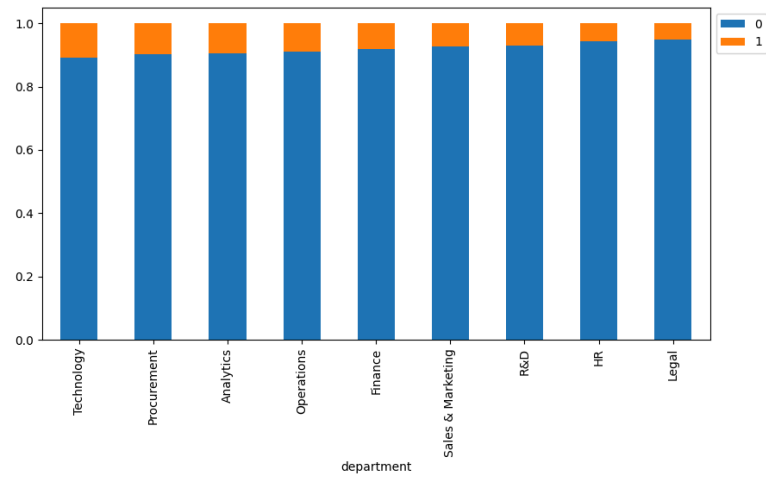
```
distribution_plot_wrt_target(data, "avg_training_score", "is_promoted") ##
Complete the code to create distribution_plot for
avg_training_score vs is_promoted
```



Target variable vs Department

```
stacked_barplot(data, "department", "is_promoted")
```

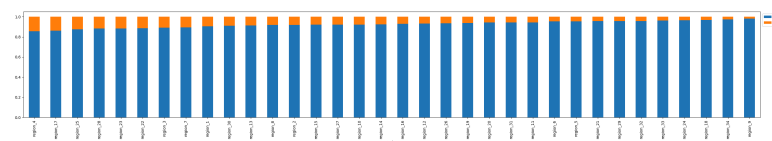
is_promoted	0	1	All
department			
All	50140	4668	54808
Sales & Marketing	15627	1213	16840
Operations	10325	1023	11348
Technology	6370	768	7138
Procurement	6450	688	7138
Analytics	4840	512	5352
Finance	2330	206	2536
HR	2282	136	2418
R&D	930	69	999
Legal	986	53	1039



Target variable vs Region

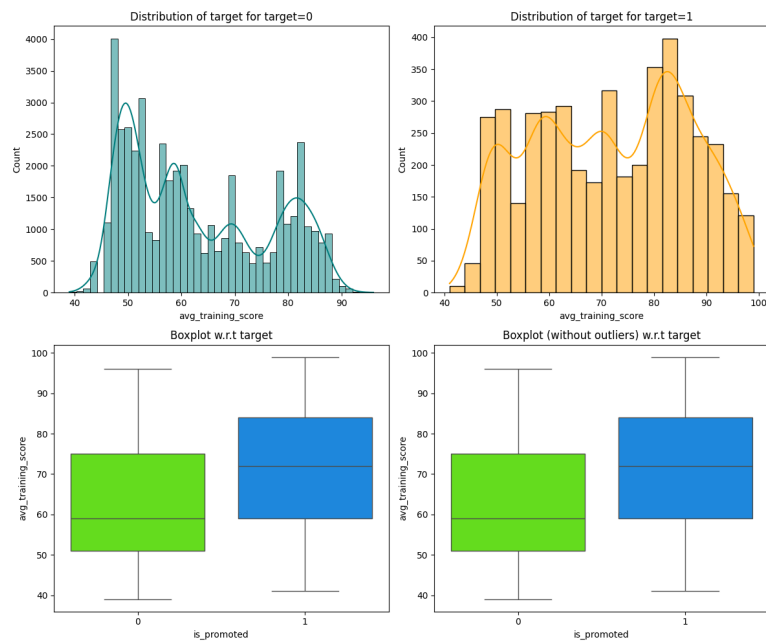
```
stacked_barplot(data, "region", "is_promoted")
```

is_promoted	0	1	All
region			
All	50140	4668	54808
region_2	11354	989	12343
region_22	5694	734	6428
region_7	4327	516	4843
region_4	1457	246	1703
region_13	2418	230	2648
region_15	2586	222	2808
region_28	1164	154	1318
region_26	2117	143	2260
region_23	1038	137	1175
region_27	1528	131	1659
region_31	1825	110	1935
region_17	687	109	796
region_25	716	103	819
region_16	1363	102	1465
region_11	1241	74	1315
region_14	765	62	827
region_30	598	59	657
region_1	552	58	610
region_8	602	53	655
region_19	821	53	874
region_10	597	51	648
region_20	801	49	850
region_29	951	43	994
region_32	905	40	945
region_3	309	37	346
region_5	731	35	766
region_12	467	33	500
region_6	658	32	690
region_21	393	18	411
region_24	490	18	508
region_33	259	10	269
region_34	284	8	292
region_9	412	8	420
region_18	30	1	31



Target variable vs Education

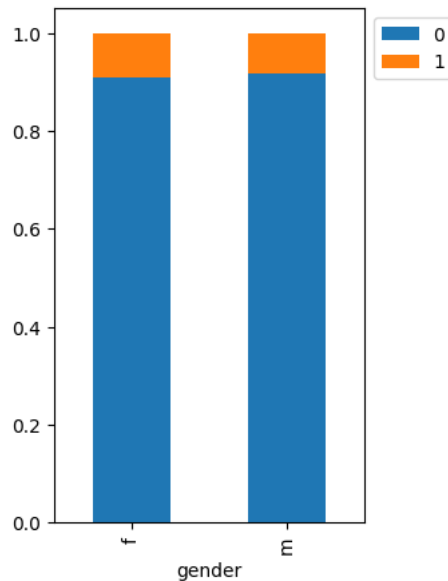
```
distribution_plot_wrt_target(data, "avg_training_score", "is_promoted") ##
# Complete the code to create distribution_plot for education vs is_promoted
```

Target variable vs Gender

`stacked_barplot(data, "gender", "is_promoted")` *## Complete the code to create distribution_plot for gender vs is_promoted*

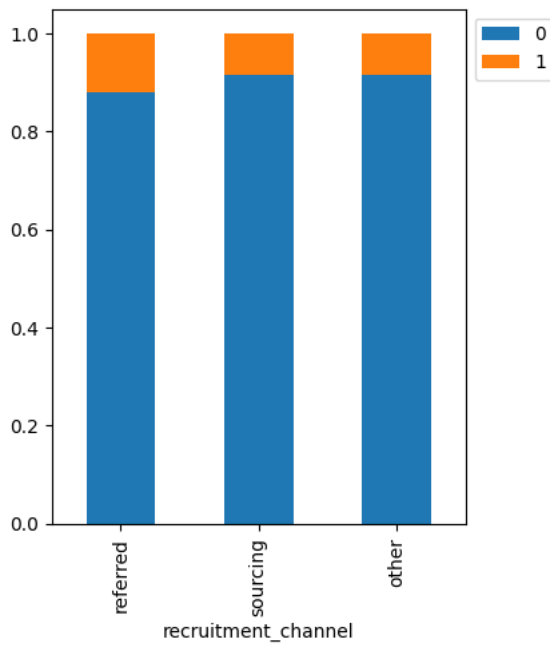
is_promoted	0	1	All
gender			
All	50140	4668	54808
m	35295	3201	38496
f	14845	1467	16312



Target variable vs Recruitment Channel

`stacked_barplot(data, "recruitment_channel", "is_promoted")` *## Complete the code to create distribution_plot for recruitment_channel vs is_promoted*

is_promoted	0	1	All
recruitment_channel			
All	50140	4668	54808
other	27890	2556	30446
sourcing	21246	1974	23220
referred	1004	138	1142

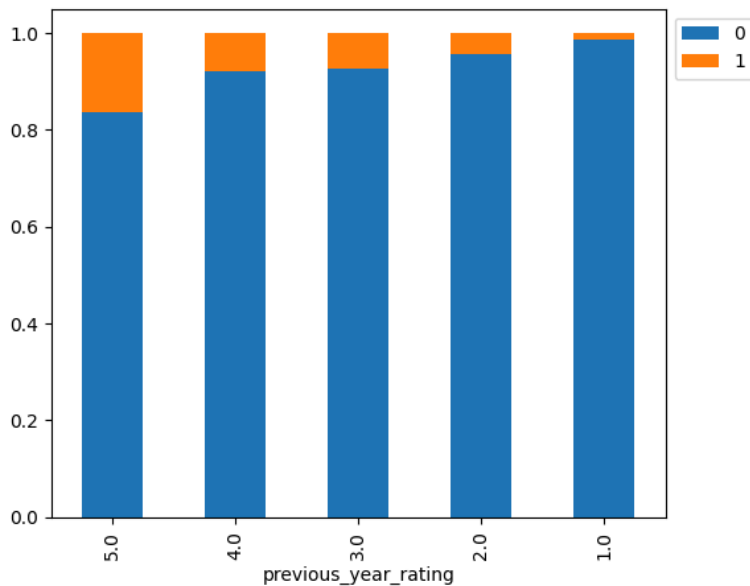


Let's see the previous rating(previous_year_rating) vary by the employee's promotion status (is_promoted)

Target variable vs Previous Year Rating

`stacked_barplot(data, "previous_year_rating", "is_promoted")` *## Complete the code to create distribution_plot for previous_year_rating vs is_promoted*

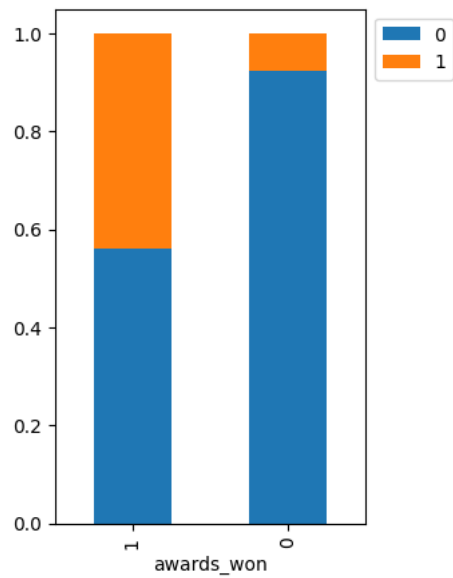
is_promoted	0	1	All
previous_year_rating			
All	46355	4329	50684
5.000	9820	1921	11741
3.000	17263	1355	18618
4.000	9093	784	9877
2.000	4044	181	4225
1.000	6135	88	6223



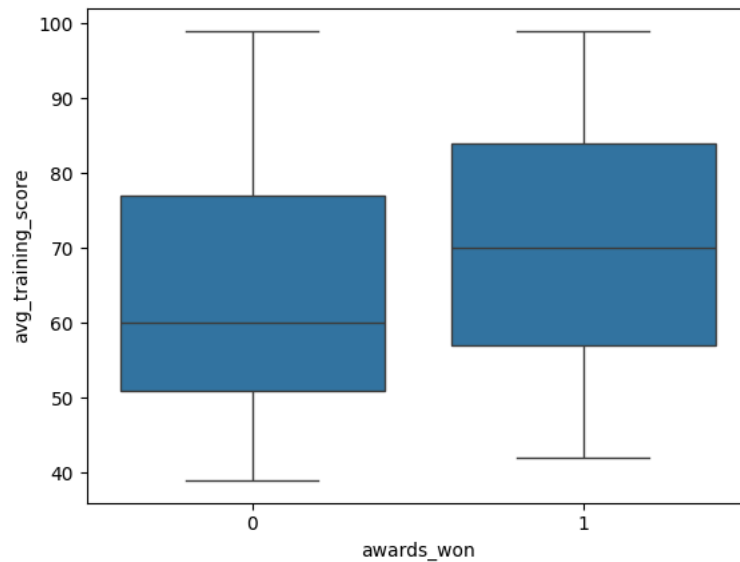
Target variable vs Awards Won

`stacked_barplot(data, "awards_won", "is_promoted")` *## Complete the code to create distribution_plot for awards_won vs is_promoted*

is_promoted	0	1	All
awards_won			
All	50140	4668	54808
0	49429	4109	53538
1	711	559	1270



```
sns.boxplot(data=data, x="awards_won", y="avg_training_score")
<Axes: xlabel='awards_won', ylabel='avg_training_score'>
```



Let's see the attributes that have a strong correlation with each other

[link text](#)### Correlation Heatmap

```
plt.figure(figsize=(15, 7))
sns.heatmap(data.corr(numeric_only=True), annot=True, vmin=-1, vmax=1,
            fmt=".2f", cmap="Spectral")
plt.show()
```



Data Preprocessing

```
data1 = data.copy()
```

Train-Test Split

```
X = data1.drop(["is_promoted"], axis=1)
y = data1["is_promoted"]

# Splitting data into training and validation set:

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.2,
                                                    random_state=1, stratify=y) ## Complete the code to split the data
                                                    into train test in the ratio 80:20

X_test, X_val, y_test, y_val = train_test_split(X_temp, y_temp,
                                                test_size=0.25, random_state=1, stratify=y_temp) ## Complete the
                                                code to split the data into train test in the ratio 75:25

print(X_train.shape, X_val.shape, X_test.shape)

(43846, 11) (2741, 11) (8221, 11)
```

Missing value imputation

```
# Defining the imputers for numerical and categorical variables
imputer_mode = SimpleImputer(strategy="most_frequent")
imputer_median = SimpleImputer(strategy="median")

# Fit and transform the train data
X_train[["education"]] = imputer_mode.fit_transform(X_train[["education"]])

# Transform the validation data
X_val[["education"]] = imputer_mode.transform(X_val[["education"]]) ##
    Complete the code to impute missing values in X_val

# Transform the test data
X_test[["education"]] = imputer_mode.transform(X_test[["education"]]) ##
    Complete the code to impute missing values in X_test

# Fit and transform the train data
X_train[["previous_year_rating", "avg_training_score"]] =
    imputer_median.fit_transform(
        X_train[["previous_year_rating", "avg_training_score"]]
    )

# Transform the validation data
X_val[["previous_year_rating", "avg_training_score"]] =
    imputer_median.transform(X_val[["previous_year_rating",
    "avg_training_score"]])

## Complete the code to impute missing values in X_val

# Transform the test data
X_test[["previous_year_rating", "avg_training_score"]] =
    imputer_median.transform(X_test[["previous_year_rating",
    "avg_training_score"]]) ## Complete the code to impute missing
    values in X_test
```

```
# Checking that no column has missing values in train, validation and test
sets
print(X_train.isna().sum())
print("-" * 30)
print(X_val.isna().sum())
print("-" * 30)
print(X_test.isna().sum())

department      0
region          0
education        0
gender          0
recruitment_channel  0
no_of_trainings  0
age             0
previous_year_rating  0
length_of_service  0
awards_won      0
avg_training_score  0
dtype: int64
-----
department      0
region          0
education        0
gender          0
recruitment_channel  0
no_of_trainings  0
age             0
previous_year_rating  0
length_of_service  0
awards_won      0
avg_training_score  0
dtype: int64
-----
department      0
region          0
education        0
gender          0
recruitment_channel  0
no_of_trainings  0
age             0
previous_year_rating  0
length_of_service  0
awards_won      0
avg_training_score  0
dtype: int64
```

Encoding categorical variables

```
X_train = pd.get_dummies(X_train, drop_first=True)
X_val = pd.get_dummies(X_val, drop_first=True) ## Complete the code to
impute missing values in X_val
X_test = pd.get_dummies(X_test, drop_first=True) ## Complete the code to
impute missing values in X_val
print(X_train.shape, X_val.shape, X_test.shape)
print(X_train.shape, X_test.shape)

(43846, 52) (2741, 52) (8221, 52)
(43846, 52) (8221, 52)
```

Building the model

Model evaluation criterion

Model can make wrong predictions as:

- Predicting an employee should get promoted when he/she should not get promoted
- Predicting an employee should not get promoted when he/she should get promoted

Which case is more important?

- Both cases are important here as not promoting a deserving employee might lead to less productivity and the company might lose a good employee which affects the company's growth. Further, giving promotion to a non-deserving employee would lead to loss of monetary resources and giving such employee higher responsibility might again affect the company's growth.

How to reduce this loss i.e need to reduce False Negatives as well as False Positives?

- Bank would want F1-score to be maximized, as both classes are important here. Hence, the focus should be on increasing the F1-score rather than focusing on just one metric i.e. Recall or Precision.

First, let's create two functions to calculate different metrics and confusion matrix, so that we don't have to use the same code repeatedly for each model.

```
# defining a function to compute different metrics to check performance of a
# classification model built using sklearn
def model_performance_classification_sklearn(model, predictors, target):
    """
    Function to compute different metrics to check classification model
    performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # predicting using the independent variables
    pred = model.predict(predictors)

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred) # to compute Recall
    precision = precision_score(target, pred) # to compute Precision
    f1 = f1_score(target, pred, average="macro") # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {
            "Accuracy": acc,
            "Recall": recall,
            "Precision": precision,
            "F1": f1,
        },
        index=[0],
    )

    return df_perf

def confusion_matrix_sklearn(model, predictors, target):
    """
    To plot the confusion_matrix with percentages

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    y_pred = model.predict(predictors)
    cm = confusion_matrix(target, y_pred)
    labels = np.asarray(
        [
            ["{0:0.0f}".format(item) + "\n{0:.2%}".format(item /
            cm.flatten().sum())]
            for item in cm.flatten()
        ]
    ).reshape(2, 2)

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=labels, fmt="")
    plt.ylabel("True label")
    plt.xlabel("Predicted label")
```

Model Building - Original Data

```
models = [] # Empty list to store all the models

# Appending models into the list'
## Complete the code to append remaining 4 models in the list models

# Original models list
models.append(("Bagging", BaggingClassifier(random_state=1)))
models.append(("Random Forest", RandomForestClassifier(random_state=1)))
models.append(("AdaBoost", AdaBoostClassifier(random_state=1)))
models.append(("Gradient Boosting", GradientBoostingClassifier(random_state=1)))
models.append(("XGBoost", XGBClassifier(random_state=1,
eval_metric="logloss")))
```

```

results1 = []
names     = []
scorer    = "f1_macro"

print("\nCross-Validation Cost:\n")
for name, model in models:          # ← models, not models_over
    kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
    cv_result = cross_val_score(estimator=model, X=X_train, y=y_train,
                                scoring=scorer, cv=kfold)
    results1.append(cv_result)
    names.append(name)
    print("{}: {}".format(name, cv_result.mean()))

print("\nValidation Performance:\n")
for name, model in models:          # ← models, not models_over
    model.fit(X_train, y_train)      # ← X_train, not X_train_over
    scores = recall_score(y_val, model.predict(X_val))
    print("{}: {}".format(name, scores))

Cross-Validation Cost:

Bagging: 0.7038584587045673
Random Forest: 0.6783220856848068
AdaBoost: 0.5964855874794006
Gradient Boosting: 0.6958158413829658
XGBoost: 0.7229737546792171

Validation Performance:

Bagging: 0.2948717948717949
Random Forest: 0.2606837606837607
AdaBoost: 0.13247863247863248
Gradient Boosting: 0.2777777777777778
XGBoost: 0.3247863247863248

# Plotting boxplots for CV scores of all models defined above
fig = plt.figure(figsize=(10, 7))

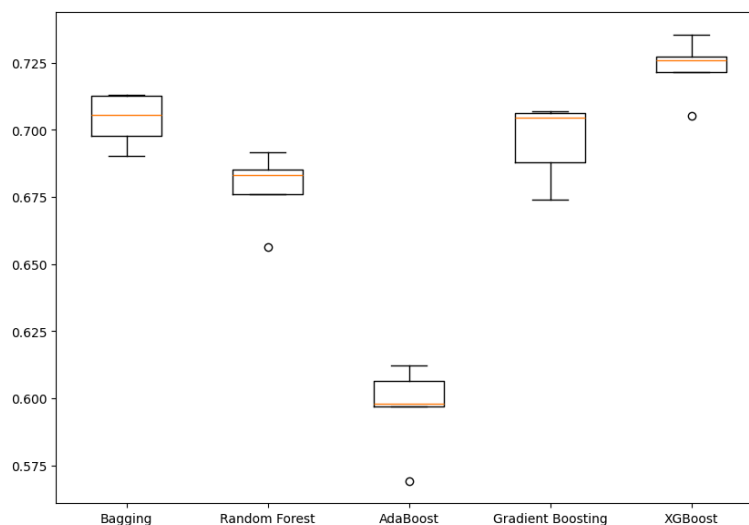
fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)

plt.boxplot(results1)
ax.set_xticklabels(names)

plt.show()

```

Algorithm Comparison



Model Building - Oversampled Data

```

print("Before Oversampling, counts of label 'Yes': {}".format(sum(y_train ==
1)))
print("Before Oversampling, counts of label 'No': {} \n".format(sum(y_train
== 0)))

```

```

sm = SMOTE(
sampling_strategy=1, k_neighbors=5, random_state=1
) # Synthetic Minority Over Sampling Technique
X_train_over, y_train_over = sm.fit_resample(X_train, y_train)

print("After Oversampling, counts of label 'Yes': {}".format(sum(y_train_over
== 1)))
print("After Oversampling, counts of label 'No': {}
\n".format(sum(y_train_over == 0)))

print("After Oversampling, the shape of train_X:
{}".format(X_train_over.shape))
print("After Oversampling, the shape of train_y: {}
\n".format(y_train_over.shape))

Before Oversampling, counts of label 'Yes': 3734
Before Oversampling, counts of label 'No': 40112

After Oversampling, counts of label 'Yes': 40112
After Oversampling, counts of label 'No': 40112

After Oversampling, the shape of train_X: (80224, 52)
After Oversampling, the shape of train_y: (80224,)

models_over = []
models_over.append(("Bagging", BaggingClassifier(random_state=1)))
models_over.append(("Random Forest",
RandomForestClassifier(random_state=1)))
models_over.append(("AdaBoost", AdaBoostClassifier(random_state=1)))
models_over.append(("Gradient
Boosting", GradientBoostingClassifier(random_state=1)))
models_over.append(("XGBoost", XGBClassifier(random_state=1,
eval_metric="logloss")))

results2 = []
names2 = []
scorer = "f1_macro"

print("\nCross-Validation Cost:\n")
for name, model in models_over:
    kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
    cv_result = cross_val_score(estimator=model, X=X_train_over,
y=y_train_over, scoring=scorer, cv=kfold)
    results2.append(cv_result)
    names2.append(name)
    print("{}: {}".format(name, cv_result.mean()))

print("\nValidation Performance:\n")
for name, model in models_over:
    model.fit(X_train_over, y_train_over)
    scores = recall_score(y_val, model.predict(X_val))
    print("{}: {}".format(name, scores)) ## Complete the code to build models
on oversampled data
## Note - Take reference from the original models built above

Cross-Validation Cost:

Bagging: 0.9512693557194716
Random Forest: 0.9603272965879741
AdaBoost: 0.8142277348274056
Gradient Boosting: 0.8809927043363077
XGBoost: 0.9407264311459684

Validation Performance:

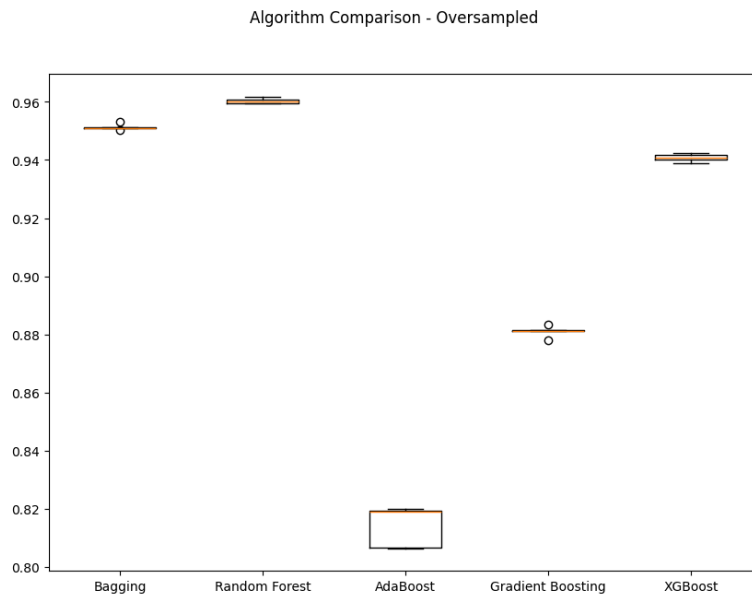
Bagging: 0.3418803418803419
Random Forest: 0.27350427350427353
AdaBoost: 0.4700854700854701
Gradient Boosting: 0.43162393162393164
XGBoost: 0.34615384615384615

# Plotting boxplots for CV scores of all models defined above
fig = plt.figure(figsize=(10, 7))
fig.suptitle("Algorithm Comparison - Oversampled")
ax = fig.add_subplot(111)
plt.boxplot(results2)

```



```
ax.set_xticklabels(names2)
plt.show() ## Write the code to create boxplot to check model performance on oversampled data
```



Model Building - Undersampled Data

```
rus = RandomUnderSampler(random_state=1)
X_train_un, y_train_un = rus.fit_resample(X_train, y_train)

print("Before Under Sampling, counts of label 'Yes': {}".format(sum(y_train
== 1)))
print("Before Under Sampling, counts of label 'No': {} \n".format(sum(y_train
== 0)))

print("After Under Sampling, counts of label 'Yes': {}".format(sum(y_train_un
== 1)))
print("After Under Sampling, counts of label 'No': {}
\n".format(sum(y_train_un == 0)))

print("After Under Sampling, the shape of train_X:
{}".format(X_train_un.shape))
print("After Under Sampling, the shape of train_y: {}
\n".format(y_train_un.shape))

Before Under Sampling, counts of label 'Yes': 3734
Before Under Sampling, counts of label 'No': 40112

After Under Sampling, counts of label 'Yes': 3734
After Under Sampling, counts of label 'No': 3734

After Under Sampling, the shape of train_X: (7468, 52)
After Under Sampling, the shape of train_y: (7468,)

models_un = []
models_un.append(("Bagging", BaggingClassifier(random_state=1)))
models_un.append(("Random Forest", RandomForestClassifier(random_state=1)))
models_un.append(("AdaBoost", AdaBoostClassifier(random_state=1)))
models_un.append(("Gradient Boosting", GradientBoostingClassifier(random_state=1)))
models_un.append(("XGBoost", XGBClassifier(random_state=1,
eval_metric="logloss")))

results3 = []
names3 = []

print("\nCross-Validation Cost:\n")
for name, model in models_un:
    kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
    cv_result = cross_val_score(estimator=model, X=X_train_un, y=y_train_un,
scoring=scorer, cv=kfold)
    results3.append(cv_result)
    names3.append(name)
    print("{}: {}".format(name, cv_result.mean()))
```

```

print("\nValidation Performance:\n")
for name, model in models_un:
    model.fit(X_train_un, y_train_un)
    scores = recall_score(y_val, model.predict(X_val))
    print("{}: {}".format(name, scores)) ## Complete the code to build models
    on undersampled data
## Note - Take reference from the original models built above

```

Cross-Validation Cost:

```

Bagging: 0.690922371059864
Random Forest: 0.707142901054085
AdaBoost: 0.652631716393266
Gradient Boosting: 0.7216343454621688
XGBoost: 0.708981139658766

```

Validation Performance:

```

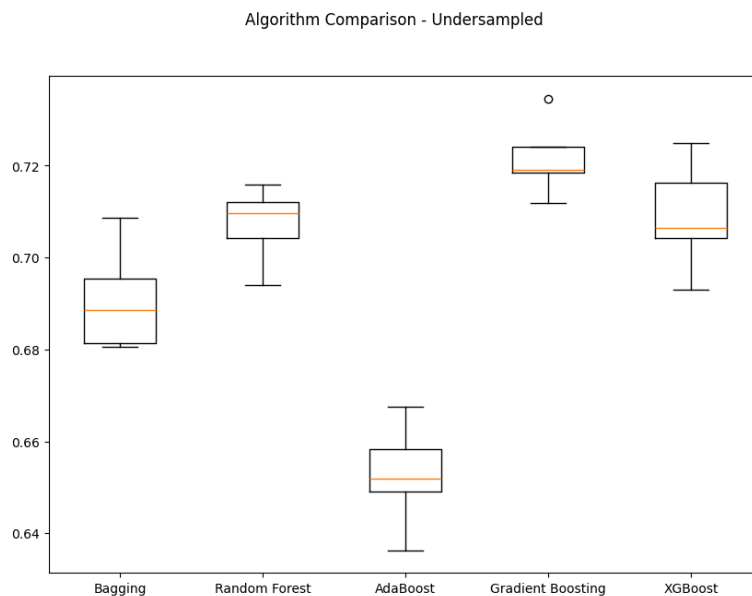
Bagging: 0.6196581196581197
Random Forest: 0.6623931623931624
AdaBoost: 0.7264957264957265
Gradient Boosting: 0.5811965811965812
XGBoost: 0.6452991452991453

```

```

# Plotting boxplots for CV scores of all models defined above
# Boxplot for undersampled models
fig = plt.figure(figsize=(10, 7))
fig.suptitle("Algorithm Comparison - Undersampled")
ax = fig.add_subplot(111)
plt.boxplot(results3)
ax.set_xticklabels(names3)
plt.show() ## Write the code to create boxplot to check model performance on
undersampled data

```



Hyperparameter Tuning

Note

1. Sample parameter grid has been provided to do necessary hyperparameter tuning. One can extend/reduce the parameter grid based on execution time and system configuration to try to improve the model performance further wherever needed.
2. The models chosen in this notebook are based on test runs. One can update the best models as obtained upon code execution and tune them for best performance.

Tuning AdaBoost using Undersampled data

```

%%time

# defining model
Model = AdaBoostClassifier(random_state=1)

```

```

# Parameter grid to pass in RandomSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "estimator": [
        DecisionTreeClassifier(max_depth=1, random_state=1),
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
}

# Type of scoring used to compare parameter combinations
scorer = 'f1_macro'

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=Model,
    param_distributions=param_grid, n_jobs = -1, n_iter=50,
    scoring=scorer, cv=5, random_state=1)

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_un, y_train_un)
## Complete the code to fit the model on undersampled data

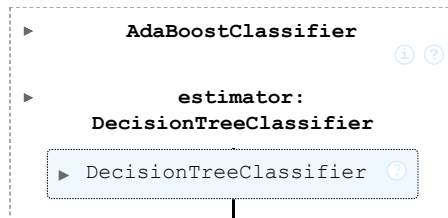
print("Best parameters are {} with CV score={}: ".format(randomized_cv.best_params_, randomized_cv.best_score_))

Best parameters are {'n_estimators': np.int64(70), 'learning_rate': 1,
'estimator': DecisionTreeClassifier(max_depth=3, random_state=1)} with CV
score=0.7092365867486399:
CPU times: user 2.72 s, sys: 169 ms, total: 2.89 s
Wall time: 2min 17s

# Creating new pipeline with best parameters
tuned_adb1 = AdaBoostClassifier( random_state=1,
    n_estimators= 70, learning_rate= 1, estimator=
        DecisionTreeClassifier(max_depth=3, random_state=1)
) ## Complete the code with the best parameters obtained from tuning

tuned_adb1.fit(X_train_un, y_train_un) ## Complete the code to fit the model
on undersampled data

```



```

adb1_train = model_performance_classification_sklearn(tuned_adb1, X_train_un,
    y_train_un)
## Complete the code to check the performance on training set
adb1_train

```

	Accuracy	Recall	Precision	F1
0	0.723	0.675	0.747	0.723

```

# Checking model's performance on validation set
adb1_val = adb1_val = model_performance_classification_sklearn(tuned_adb1,
    X_val, y_val) ## Complete the code to check the performance on
validation set
adb1_val

```

	Accuracy	Recall	Precision	F1
0	0.764	0.645	0.211	0.588

Tuning AdaBoost using original data

```

%%time

# defining model
Model = AdaBoostClassifier(random_state=1)

```

```

# Parameter grid to pass in RandomSearchCV
param_grid = {
    "n_estimators": np.arange(10, 110, 10),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "estimator": [
        DecisionTreeClassifier(max_depth=1, random_state=1),
        DecisionTreeClassifier(max_depth=2, random_state=1),
        DecisionTreeClassifier(max_depth=3, random_state=1),
    ],
}

# Type of scoring used to compare parameter combinations
scorer = 'f1_macro'

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=Model,
    param_distributions=param_grid, n_jobs = -1, n_iter=50,
    scoring=scorer, cv=5, random_state=1)

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train, y_train) ## Complete the code to fit the model on
original data

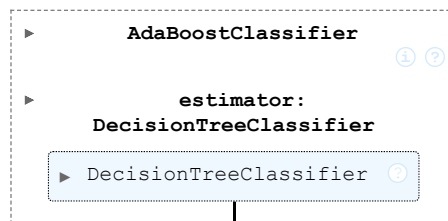
print("Best parameters are {} with CV score={};"
    .format(randomized_cv.best_params_,randomized_cv.best_score_))

Best parameters are {'n_estimators': np.int64(70), 'learning_rate': 1,
'estimator': DecisionTreeClassifier(max_depth=3, random_state=1)} with CV
score=0.6469262291040385:
CPU times: user 8.26 s, sys: 739 ms, total: 9 s
Wall time: 10min 33s

# Creating new pipeline with best parameters
tuned_adb2 = AdaBoostClassifier(random_state=1, n_estimators=100,
    learning_rate =0.1,
    estimator=DecisionTreeClassifier(max_depth=3, random_state=1))

tuned_adb2.fit(X_train, y_train) ## Complete the code to fit the model on
original data

```



```

adb2_train = model_performance_classification_sklearn(tuned_adb2, X_train,
    y_train)
## Complete the code to check the performance on training set
adb2_train

```

	Accuracy	Recall	Precision	F1
0	0.925	0.122	0.946	0.589

```

# Checking model's performance on validation set
adb2_val = model_performance_classification_sklearn(tuned_adb2, X_val, y_val)
## Complete the code to check the performance on validation set
adb2_val

```

	Accuracy	Recall	Precision	F1
0	0.925	0.128	0.938	0.593

Tuning Gradient Boosting using undersampled data

```

%%time

#Creating pipeline
Model = GradientBoostingClassifier(random_state=1)

#Parameter grid to pass in RandomSearchCV
param_grid = {

```

```

    "init":
        [AdaBoostClassifier(random_state=1),DecisionTreeClassifier(random_state=1)],
    "n_estimators": np.arange(75,150,25),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "subsample":[0.5,0.7,1],
    "max_features":[0.5,0.7,1],
}

```

```

# Type of scoring used to compare parameter combinations
scorer = 'f1_macro'

```

```

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=Model,
    param_distributions=param_grid, n_iter=50, scoring=scorer, cv=5,
    random_state=1, n_jobs = -1)

```

```

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train_un, y_train_un) ## Complete the code to fit the
    model on under sampled data

```

```

print("Best parameters are {} with CV score={}:\"
    .format(randomized_cv.best_params_,randomized_cv.best_score_)

```

```

Best parameters are {'subsample': 0.7, 'n_estimators': np.int64(100),
'max_features': 0.5, 'learning_rate': 0.1, 'init':
AdaBoostClassifier(random_state=1)} with CV score=0.7207387245467136:
CPU times: user 2.87 s, sys: 214 ms, total: 3.09 s
Wall time: 2min 47s

```

```

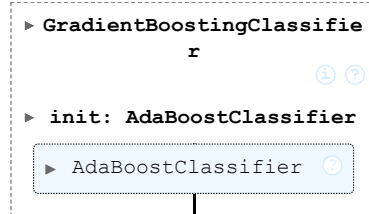
# Creating new pipeline with best parameters
tuned_gbm1 = GradientBoostingClassifier(
    max_features=0.8,
    init=AdaBoostClassifier(random_state=1),
    random_state=1,
    learning_rate=0.1,
    n_estimators=150,
    subsample=0.9)## Complete the code with the best parameters obtained from
    tuning

```

```

tuned_gbm1.fit(X_train_un, y_train_un)

```



```

gbm1_train = model_performance_classification_sklern(tuned_gbm1, X_train_un,
    y_train_un) ## Complete the code to check the performance on
    oversampled train set
gbm1_train

```

	Accuracy	Recall	Precision	F1
0	0.755	0.680	0.801	0.754

```

gbm1_val = model_performance_classification_sklern(tuned_gbm1, X_val,
    y_val) ## Complete the code to check the performance on validation
    set
gbm1_val

```

	Accuracy	Recall	Precision	F1
0	0.783	0.615	0.222	0.599

Tuning Gradient Boosting using original data

```

%%time

```

```

#defining model

```

```

Model = GradientBoostingClassifier(random_state=1)

```

```

#Parameter grid to pass in RandomSearchCV

```

```

param_grid = {
    "init":
        [AdaBoostClassifier(random_state=1),DecisionTreeClassifier(random_state=1)],
    "n_estimators": np.arange(75,150,25),
    "learning_rate": [0.1, 0.01, 0.2, 0.05, 1],
    "subsample":[0.5,0.7,1],
    "max_features":[0.5,0.7,1],
}

```

```

# Type of scoring used to compare parameter combinations
scorer = 'f1_macro'

```

```

#Calling RandomizedSearchCV
randomized_cv = RandomizedSearchCV(estimator=Model,
    param_distributions=param_grid, n_iter=50, scoring=scorer, cv=5,
    random_state=1, n_jobs = -1)

```

```

#Fitting parameters in RandomizedSearchCV
randomized_cv.fit(X_train, y_train) ## Complete the code to fit the model on
original data

```

```

print("Best parameters are {} with CV score={}:\"
    .format(randomized_cv.best_params_,randomized_cv.best_score_))

```

```

Best parameters are {'subsample': 0.7, 'n_estimators': np.int64(125),
' max_features': 0.7, 'learning_rate': 0.2, 'init':
AdaBoostClassifier(random_state=1)} with CV score=0.7194607575241472:
CPU times: user 11.7 s, sys: 682 ms, total: 12.4 s
Wall time: 12min 17s

```

```

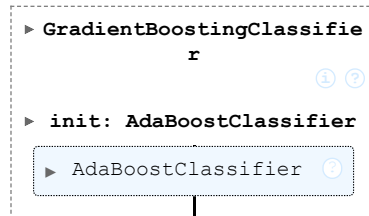
# Creating new pipeline with best parameters
tuned_gbm2 = GradientBoostingClassifier(
    max_features=0.7,
    init=AdaBoostClassifier(random_state=1),
    random_state=1,
    learning_rate=0.1,
    n_estimators=125,
    subsample=1,
)## Complete the code with the best parameters obtained from tuning

```

```

tuned_gbm2.fit(X_train, y_train)

```



```

gbm2_train = model_performance_classification_sklearn(tuned_gbm2, X_train,
    y_train) ## Complete the code to check the performance on original
data

```

```

gbm2_train

```

	Accuracy	Recall	Precision	F1
0	0.938	0.284	0.953	0.702

```

gbm2_val = model_performance_classification_sklearn(tuned_gbm2, X_val,
    y_val) ## Complete the code to check the performance on validation
set

```

```

gbm2_val

```

	Accuracy	Recall	Precision	F1
0	0.937	0.278	0.956	0.699

Model Comparison and Final Model Selection

```

# training performance comparison

```

```

models_train_comp_df = pd.concat(
    [

```

```

        gbm1_train.T,
        gbm2_train.T,
        adb1_train.T,
        adb2_train.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "Gradient boosting trained with Undersampled data",
    "Gradient boosting trained with Original data",
    "AdaBoost trained with Undersampled data",
    "AdaBoost trained with Original data",
]
print("Training performance comparison:")
models_train_comp_df

```

Training performance comparison:

	Gradient boosting trained with Undersampled data	Gradient boosting trained with Original data	AdaBoost trained with Undersampled data	AdaBoost trained with Original data
Accuracy	0.755	0.938	0.723	0.925
Recall	0.680	0.284	0.675	0.122
Precision	0.801	0.953	0.747	0.946
F1	0.754	0.702	0.723	0.589

validation performance comparison

```

models_val_comp_df = pd.concat(
    [gbm1_val.T, gbm2_val.T, adb1_val.T, adb2_val.T], axis=1)
models_val_comp_df.columns = [
    "Gradient Boosting - Undersampled",
    "Gradient Boosting - Original",
    "AdaBoost - Undersampled",
    "AdaBoost - Original",
]
print("Validation performance comparison:")
models_val_comp_df ## Write the code to compare the performance on validation set

```

Validation performance comparison:

	Gradient Boosting - Undersampled	Gradient Boosting - Original	AdaBoost - Undersampled	AdaBoost - Original
Accuracy	0.783	0.937	0.764	0.925
Recall	0.615	0.278	0.645	0.128
Precision	0.222	0.956	0.211	0.938
F1	0.599	0.699	0.588	0.593

Now we have our final model, so let's find out how our final model is performing on unseen test data.

Let's check the performance on test set
model_performance_classification_sklern(tuned_gbm2, X_test, y_test) *## Write the code to check the performance of best model on test data*

	Accuracy	Recall	Precision	F1
0	0.939	0.301	0.930	0.711

Feature Importances

```

feature_names = X_train.columns
best_model = tuned_gbm1 # or whichever model has the best F1 on validation set
model_performance_classification_sklern(best_model, X_test, y_test)

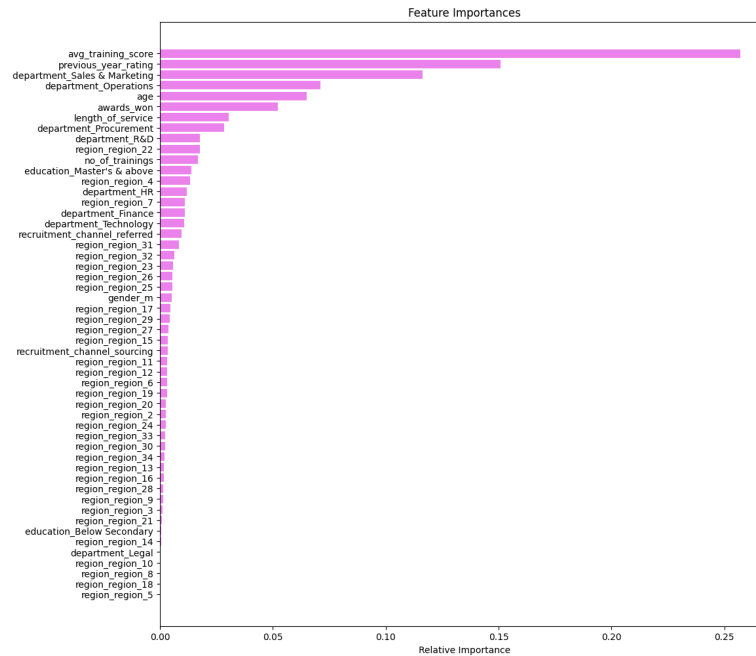
```

```

importances = best_model.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(12, 12))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet",
         align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()

```



Business Insights and Conclusions

Key Business Insights

Merit Over Seniority: Age and length of service have almost zero correlation with promotion (-0.017 and -0.010).

Impact of Awards: Winning an award is the strongest individual predictor, boosting promotion rates from 7.67% (no award) to 44.02% (with award).

Upskilling Drives Success: Promoted employees scored significantly higher on average training scores (71.84) than non-promoted peers (62.95).

Performance Baseline: Past year ratings are critical; employees rated 5.0 achieved a 16.36% promotion rate, while those rated 1.0 dropped to just 1.41%.

Departmental Differences: Promotion velocity varies widely by function, led by Technology (10.76%) and Procurement (9.64%), while Legal (5.10%) and HR (5.62%) remain the most conservative.

High-Value Sourcing: Candidates arriving through referrals hit a much higher promotion rate (12.08%) compared to standard sourcing channels (~8.4%).

Conclusions:

Data Imbalance Challenge: Only 8.52% of the workforce is promoted. Because of this high class imbalance, relying strictly on "Accuracy" is misleading and will mask poor performance on the minority class.

Optimal Model Approach: Models trained with undersampling (e.g., `tuned_gbm1`) or tuned explicitly for `f1_macro` are preferred. They successfully minimize false negatives, ensuring high-potential talent is not overlooked.

Ensemble Robustness: Tree-based algorithms (Gradient Boosting and AdaBoost) proved optimal due to their inherent ability to handle high-cardinality features (like region and department) and complex, non-linear interactions between variables.
